COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE010- Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	Oct 15, 2026	**DATE SUBMITTED:**	Oct 19, 2026
NAME:	Ignacio, Guillian Carlo V. Regala, Joaquin Luiz	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 13.1 [Multithreading]

1. PROCEDURE OUTPUT

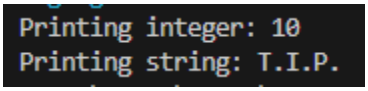
ILO A	
Screenshot	
Analysis (Ignacio)	As we can see here in the output, the first to be printed is the integer then the string. Single threading executes the lines one by one sequentially on a single CPU core. We also used a callable function pointer to execute it.
Analysis (Regala)	This program uses a separate thread to print the integer 10 and the string T.I.P.. The join function makes the main thread wait until the task is finished.

Table 13-1. Simple One-Threaded Example

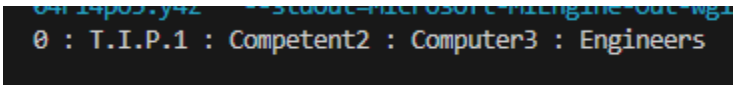
ILO B	
Screenshot Analysis	
Analysis (Ignacio)	For the output of this task, it could be different every run, because the code is joining threads randomly and whatever runs first will be first to be printed out. In multithreading, we have to join different threads together then we need to see which runs first, this is useful when we want to run 2 things at the same time.
Analysis (Regala)	The program here uses multiple threads to print different data at the same time. With this it makes the process faster while only using 1 thread.

Table 13-2. Multithreaded Example.

COMPUTER ENGINEERING

QUEZON CITY

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Part A: Demonstrate an understanding of parallelism, concurrency, and multithreading in C++ by answering the given questions. Use of supplementary materials to support answers must be cited as reference.

Ignacio	Regala
1. Write a definition of multithreading and its advantages/disadvantages. Multithreading is where we join different threads in order to run them at the same time it may use more CPU resources but it saves us time. Advantages: - Executes multiple tasks at once utilizing the CPU - Shares the same memory space Disadvantages: - There are race conditions that are complex to debug - Overstacking the CPU	**1. Write a definition of multithreading and its advantages/disadvantages.** Multithreading runs multiple threads at once. Advantages: Faster execution Disadvantage: complex coding and more errors
2. Rationalize the use of multithreading by providing at least 3 use-cases. - Multithreading is used when background apps or tabs are opened so crashes don't happen - Video games handle different types of background threads while the user only experiences the main thread so it doesn't crash his/her gaming experience. - Data Processing, when capturing different types of data that is ongoing at the same time, multithreading exists so we can capture whatever comes first.	**2. Rationalize the use of multithreading by providing at least 3 use-cases.** - Gaming: Handles graphic audio, and game logic at the same time for smoother gameplay - Web servers: Allows multiple users to access a website at the same time - Video editing: Processes video, audio, and effects faster by using multiple threads
3. Differentiate between parallelism and concurrency. - Concurrency is some kind of control flow in order to manage 2 queues in a one queue system - Pararellism creates another queue for a 2 queue system so 2 threads exist	**3. Differentiate between parallelism and concurrency.** Concurrency handles multiple tasks over overlapping periods, while parallelism runs tasks at the same time

Part B: Create C++ Code and show a solution that satisfies the given requirements below.

- **Create a global variable of type integer.**
- **Create an add function that will take an integer parameter and add that value to the global variable.**
- **Use multi-threading techniques to create 3 threads; individually pass the add function to the threads.**
- **Display the value of global variables at different combinations of using the join() per thread. Such that:**
  - **o Display**
  - **o T1.join()**
  - **o Display**
  - **o T2.join()**
  - **o Display**
  - **o T3.join()**
- **Provide an analysis based on the outputs of the multi-threading exercise.**

Screenshot of the code

```cpp
#include <iostream>
#include <thread>

int globalVariable = 0;

// Function that adds a value to the global variable
void add(int value)
{
    globalVariable += value;
}

int main()
{
    std::thread T1(add, 10);
    std::thread T2(add, 20);
    std::thread T3(add, 30);

    std::cout << "Initial value: " << globalVariable << std::endl;

    T1.join();
    std::cout << "After T1.join(): " << globalVariable << std::endl;

    T2.join();
    std::cout << "After T2.join(): " << globalVariable << std::endl;

    T3.join();
    std::cout << "After T3.join(): " << globalVariable << std::endl;

    return 0;
}
```

Screenshot of the output

```
[Running] cd "c:\Users\regal\OneDrive\Pictures\VSC
Initial value: 0
After T1.join(): 60
After T2.join(): 60
After T3.join(): 60
```

Analysis	
Ignacio	As we can see the three threads adds a value to the global variable where the initial value is 0 then once T1-T3 joins the values are supposed to be 10,30,60 depending on what the thread actually outputs first. The join function waits for each thread to finish.
Regala	The program uses three threads to add values to a global variable. The initial value is 0, then it becomes 10, 30, and finally 60. The join function makes the main program wait for each thread to finish. The mutex and condition variable ensure that the threads update the variable in the correct order and prevent data races.

Part C: Use multi-threading with one of the algorithms previously developed in the course; provide an analysis of the result.

Screenshot of Code(From HOA8.1 Ignacio's code)

```
// MERGE SORT
template <typename T>
void merge(T arr[], int left, int mid, int right){
    int n1 = mid - left + 1;
    int n2 = right - mid;

    //Temporary Arrays
    T* Lt = new T[n1];
    T* Rt = new T[n2];

    //copy data to the temporary arrays
    for (int i = 0; i < n1; i++){
        Lt[i] = arr[left + i];
    }
    for (int j = 0; j < n2; j++){
        Rt[j] = arr[mid + 1 + j];
    }
```

```
    // Merge them back
    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2){
        if (Lt[i] <= Rt[j]){
            arr[k++] = Lt[i++];
        }
        else{
            arr[k++] = Rt[j++];
        }
    }

    // copy remaining elements
    while (i < n1){
        arr[k++] = Lt[i++];
    }
    while(j < n2){
        arr[k++] = Rt[j++];
    }
```

```
    //delete memory
    delete[] Lt;
    delete[] Rt;
}
```

```
template <typename T>
void mergeSort(T arr[], int left, int right){
    if (left < right){
        int mid = (left + right) / 2;

        // sort first and the second half
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        // merge them back
        merge(arr, left, mid, right);
    }
}
```

OUTPUT:

```
Merge sort:
0 2 2 3 4 4 5 7 10 11 12 14 16 17 18 18 19 20 24 24 25 25 26 26 29 29 29 32 34 35 35 35
 35 39 39 39 42 46 47 48 49 49 50 51 51 52 52 52 52 52 54 54 54 55 56 57 59 59 60 60 63
 63 64 64 65 66 66 66 68 68 68 69 69 70 70 71 72 74 75 76 76 77 77 79 80 81 83 84 85 86
 88 90 94 94 95 95 97 99 99 99
```

ANALYSIS FOR PART C
Ignacio
The code uses merge sort to arrange the integers to lowest to highest. Using multithreading we can see what array can finish first with the given runtime and resources of the CPU for the execution time. The output will show that the numbers are sorted correctly but the way they were sorted had different times.
Regala
The program uses Merge Sort to arrange numbers in ascending order. Through multithreading, different parts of the array can be sorted concurrently, potentially reducing execution time. The output shows that the numbers were sorted correctly. However, the actual speed improvement depends on how the threads are implemented and managed.

CONCLUSION
Ignacio
I therefore conclude that multithreading is very important in our technology today, since there are a lot of things going on in the background and multithreading helps smooth things out. Through this activity I have learned 3 ways on how to implement multithreading but I have little idea how to maximize them in different situations
Regala
In this activity, we learned about parallelism, concurrency and multithreading. We learned how to use threads to perform different tasks and how join function helps the code wait for thread to finish. Through the activities we performed we also saw how multithreading can make programs more efficient, but it needs to be implemented properly to avoid errors. Overall, we were able to understand the basic concepts and apply them in our code.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **None**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

COMPUTER ENGINEERING

QUEZON CITY

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.